

TRAFFIC_CRASH – DataLab Hub

Descripción General del Proyecto

TRAFFIC_CRASH es una aplicación de análisis de datos desarrollada como proyecto final de la asignatura Programación de Computadores de la Universidad Nacional de Colombia. El objetivo principal del proyecto es explorar, analizar y visualizar información relacionada con accidentes de tránsito mediante una interfaz gráfica intuitiva desarrollada en Python.

El proyecto fue construido siguiendo la metodología incremental propuesta por DataLab. Durante las primeras etapas se implementaron funcionalidades básicas de búsqueda y análisis en consola, mientras que en la entrega final se integraron dichas funcionalidades en una interfaz gráfica utilizando PyQt5, manteniendo una separación clara entre la lógica de análisis y la presentación visual.

El conjunto de datos utilizado contiene información relacionada con vehículos involucrados en accidentes de tránsito. A partir de estos datos el sistema permite realizar búsquedas, obtener estadísticas descriptivas, comparar grupos de datos, visualizar distribuciones mediante gráficos y exportar resultados para análisis posteriores.

Además de cumplir los requisitos académicos establecidos, el proyecto busca demostrar cómo los conceptos fundamentales de programación pueden combinarse con herramientas modernas de análisis de datos para construir una aplicación funcional y útil.

Objetivos del Proyecto

Objetivo General

Desarrollar una herramienta de análisis de datos que permita explorar y visualizar información relacionada con accidentes de tránsito mediante una interfaz gráfica amigable.

Objetivos Específicos

- Implementar funciones de búsqueda sobre el dataset.
 - Calcular estadísticas descriptivas de los datos.
 - Organizar la información utilizando estructuras de datos adecuadas.
 - Registrar y almacenar consultas realizadas por los usuarios.
 - Generar visualizaciones que faciliten la interpretación de los datos.
 - Exportar resultados para su uso posterior.
 - Aplicar los conceptos vistos durante el curso en un proyecto real.
-

Tecnologías Utilizadas

Para el desarrollo del proyecto se utilizaron diversas librerías y herramientas que permitieron construir una aplicación robusta y organizada.

Python

Python fue utilizado como lenguaje principal debido a su facilidad para el procesamiento de datos y el desarrollo de interfaces gráficas.

Pandas

La librería Pandas se empleó para cargar y manipular el dataset de manera eficiente. Gracias a esta herramienta fue posible trabajar con miles de registros utilizando estructuras tipo DataFrame.

Principales ventajas obtenidas:

- Lectura rápida de archivos CSV.
- Filtrado eficiente de registros.
- Cálculo simplificado de estadísticas.
- Organización estructurada de la información.

Matplotlib

Matplotlib fue utilizado para generar gráficos estadísticos que permiten visualizar patrones presentes en los datos.

Los gráficos incluidos en la aplicación permiten interpretar la información de forma más intuitiva que mediante tablas o texto.

PyQt5

PyQt5 fue utilizado para construir toda la interfaz gráfica del sistema.

Gracias a esta librería se implementaron:

- Ventana principal.
 - Botones de interacción.
 - Campos de texto.
 - Áreas de resultados.
 - Integración de gráficos.
-

Arquitectura General del Sistema

El proyecto sigue una arquitectura modular donde cada archivo tiene una responsabilidad específica.

Esta separación facilita la lectura del código, mejora el mantenimiento del proyecto y cumple con los principios de diseño recomendados para aplicaciones de software.

La arquitectura puede resumirse de la siguiente manera:

Usuario → Interfaz Gráfica → Módulos de Análisis → Archivos de Datos

De esta forma la interfaz únicamente solicita operaciones y muestra resultados, mientras que toda la lógica de procesamiento se encuentra en módulos independientes.

Descripción Detallada de los Archivos

main.py

Este archivo constituye el punto de entrada del programa.

Cuando el usuario ejecuta la aplicación, el sistema inicia desde este archivo.

Sus responsabilidades principales son:

- Crear la aplicación PyQt5.
- Inicializar la ventana principal.
- Ejecutar el ciclo de eventos de la interfaz.
- Mantener separada la lógica de análisis.

La existencia de este archivo simplifica el lanzamiento de la aplicación y permite que futuras modificaciones no afecten directamente el resto de módulos.

analisis.py

Este módulo contiene todas las funciones encargadas de procesar los datos.

Representa el núcleo analítico del proyecto.

Carga de Datos

Las funciones de carga permiten leer el dataset y convertirlo en estructuras de datos manipulables por Python.

Una vez cargados los datos, todas las operaciones posteriores se realizan sobre dichas estructuras.

Función de Búsqueda

La búsqueda permite localizar registros específicos dentro del conjunto de datos.

Proceso:

1. El usuario introduce un término.
2. El sistema recorre los registros.
3. Se identifican coincidencias.
4. Se devuelve la lista de resultados.

Esta funcionalidad resulta útil para localizar rápidamente información específica sin necesidad de revisar manualmente todo el dataset.

Función de Estadísticas

Permite calcular medidas descriptivas sobre variables numéricas.

Entre ellas:

- Valor máximo.
- Valor mínimo.
- Promedio.

Estas métricas permiten comprender mejor el comportamiento general de los datos.

Función de Agrupación

Organiza los registros según categorías determinadas.

Por ejemplo:

- Tipo de vehículo.
- Marca.
- Categoría de accidente.

El resultado muestra cuántos registros pertenecen a cada grupo.

Comparación de Grupos

Permite analizar diferencias entre dos categorías seleccionadas por el usuario.

Esta funcionalidad facilita identificar tendencias y comportamientos particulares dentro del dataset.

archivos.py

Este módulo administra todas las operaciones relacionadas con almacenamiento de información.

Su propósito es garantizar que los resultados puedan conservarse entre diferentes sesiones de uso.

Guardado de Resultados

Permite exportar resultados en formato CSV.

Esto facilita compartir información o analizarla posteriormente mediante otras herramientas.

Exportación JSON

Algunos resultados pueden almacenarse en formato JSON para conservar estructuras más complejas.

Historial de Consultas

Cada búsqueda realizada queda registrada automáticamente.

El historial almacena:

- Fecha.
- Hora.
- Consulta realizada.
- Número de resultados obtenidos.

Gracias a esto es posible reconstruir la actividad realizada por los usuarios dentro del sistema.

interfaz.py

Este archivo contiene toda la interfaz gráfica del proyecto.

Es el módulo con el que interactúa directamente el usuario.

La interfaz fue diseñada para ser sencilla e intuitiva.

Ventana Principal

Al iniciar la aplicación se muestra una ventana central desde donde pueden ejecutarse todas las funcionalidades.

Búsquedas

El usuario puede ingresar términos y visualizar inmediatamente los resultados encontrados.

Estadísticas

Mediante un botón específico se calculan y presentan estadísticas descriptivas.

Historial

Permite consultar búsquedas realizadas previamente.

Agrupaciones

Muestra la distribución de registros según categorías relevantes.

Exportación

Permite guardar resultados generados durante la sesión.

Gráficos

La interfaz incorpora gráficos desarrollados con Matplotlib para representar visualmente los datos analizados.

Visualizaciones Implementadas

Uno de los principales aportes de la entrega final es la incorporación de gráficos.

Gráfico de Barras

Este gráfico muestra la cantidad de registros asociados a cada categoría.

Su objetivo es identificar rápidamente cuáles grupos presentan mayor frecuencia dentro del dataset.

Histograma

Permite observar cómo se distribuyen los valores de una variable numérica.

Gracias a esta visualización es posible detectar concentraciones de datos y tendencias generales.

Flujo de Funcionamiento del Sistema

El funcionamiento general del proyecto puede describirse mediante los siguientes pasos:

1. El usuario ejecuta la aplicación.

2. Se inicia la interfaz gráfica.
 3. El dataset es cargado en memoria.
 4. El usuario selecciona una funcionalidad.
 5. La interfaz envía la solicitud al módulo de análisis.
 6. El módulo procesa la información.
 7. Los resultados son devueltos a la interfaz.
 8. Los resultados se muestran en pantalla.
 9. Opcionalmente se almacenan en archivos CSV o JSON.
 10. Las consultas quedan registradas en el historial.
-

Resultados Obtenidos

El sistema desarrollado permite analizar grandes cantidades de información de manera rápida y organizada.

Entre los principales resultados obtenidos se encuentran:

- Automatización de búsquedas.
- Obtención inmediata de estadísticas.
- Comparación de categorías.
- Generación de gráficos.
- Exportación de resultados.
- Persistencia de información mediante historial.

Estas características convierten al proyecto en una herramienta útil para la exploración y comprensión de datos.

Conclusiones

El desarrollo de TRAFFIC_CRASH permitió aplicar de manera práctica los conceptos aprendidos durante el curso de Programación de Computadores.

La implementación modular facilitó la organización del código y permitió separar claramente la lógica de análisis de la interfaz gráfica.

La utilización de Pandas mejoró significativamente la eficiencia en el procesamiento de datos, mientras que Matplotlib permitió transformar información numérica en representaciones visuales comprensibles.

La incorporación de PyQt5 hizo posible construir una interfaz amigable que facilita la interacción con el sistema y mejora la experiencia del usuario.

Finalmente, el proyecto demuestra cómo los fundamentos de programación, estructuras de datos, manejo de archivos, programación modular y librerías externas pueden integrarse para desarrollar una aplicación completa de análisis de datos.

Como trabajo futuro, podrían añadirse filtros avanzados, análisis predictivos, gráficos interactivos y nuevas métricas que permitan obtener conclusiones aún más profundas sobre los accidentes de tránsito analizados.